Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

Challenge: searching

Today
Welcome to your next lesson: Challenge: searching.

I'm preparing your personalized lesson based on what you've already learned, what's next for you, and the best ways to reinforce your skills.

Let's dive in — every step forward matters.

Ron, this lesson is a 6-level search challenge 😄

No new teaching in between — just your answers, then a summary at the end.

Let's start with Level 1.

Imagine a short Python list:

Python

```
nums = [4, 9, 2, 7, 5]
target = 7
```

You use linear search (walk from left to right, checking each element in order).

Question:

What index will linear search return when it finds target?

Just give me the index number (like 0, 1, 2, …).
3

Nice ✔

Index 3 is correct for 7 in that list.

Level 2 🔍

You still have:

Python

```
nums = [4, 9, 2, 7, 5]
target = 10
```

You run a linear search over the whole list.

Question:

What should the search return when the target is not found?

Answer in Python style, e.g. None, -1, or something else — your choice, but be explicit.

-1

Good choice 👍

Many implementations use -1 to mean "not found."

Level 3 🧠

Now we switch to binary search.

You have a sorted list:

Python

```
nums = [3, 5, 8, 12, 20, 31, 42]
target = 20
Using binary search (with low, high, mid indexes), what index should be returned for target 20?
```

Reply with just the index.

4

Correct again 🎯

Index 4 is where 20 lives in that sorted list.

Level 4 ⚔️

Same sorted list:

Python

```
nums = [3, 5, 8, 12, 20, 31, 42]
target = 6
You run binary search for 6.
```

Question (conceptual):

After the algorithm finishes, what should it report?

You can answer in words (like "not found") and/or say what special value you'd return (for example, -1 or None).

not found as return value for me would be -1

Perfect, that's exactly what a clean binary search should do here ☑

Level 5 ☸

You have this unsorted list:

Python

```
nums = [14, 3, 27, 9, 18, 2, 30]
You need to search this list one single time for a target value.
```

Which search approach makes more sense here?

Linear search over the list as it is
Binary search

Just answer: linear search or binary search (no explanation needed).

linear

Nice choice — linear search is the right call for a one-off search on an unsorted list 👍

Level 6 (last one) 🚀

You have a very large, sorted list of a million user IDs in Python:

Python

```
user_ids = [...]  # 1,000,000 sorted integers
```

You'll be searching this list many times per second for different IDs.

Which search approach is better overall here?

Just answer: linear search or binary search.

binary search

Binary search is exactly right here 🔥

You just cleared all 6 levels of the search challenge:

You nailed indexes vs values in both linear and binary search.
You correctly chose when to use linear search (unsorted, one□off).
You correctly chose binary search for large, sorted data with many searches.
💡 Tiny tip going forward:
When you see a search problem, pause and ask yourself two questions in order:

"Is the data sorted?"
"Am I searching once, or many times?"
Your answers will usually tell you right away whether linear or binary search makes more sense.

Ron, you've fully mastered this lesson's goals 🎯
Next up, we'll keep building on these skills. When you're ready, hit Next lesson in the Learn tab, and I'll meet you there.